PARTEA II · MULTIMEDIA ÎN CONTEXT WEB

# 4. JavaScript și DOM

Limbajul JavaScript modern, accesul la DOM, tratarea evenimentelor, temporizarea, execuția asincronă cu Promise și async/await și comunicarea cu serverul prin Fetch API.

AUTOR **conf. dr. Cristian Ioniță** · DATA **2026-10-11** · VERSIUNE **0.2**

Notă privind utilizarea IAG. Structura și conținutul prezentării aparțin autorului. Instrumentul Anthropic Claude (Claude Code, 2026) a fost utilizat, sub îndrumarea autorului, pentru formatarea în identitatea ASE, redesenarea diagramelor, transcrierea formulelor și corecturi lingvistice, precum și pentru redactarea unui număr redus de slide-uri noi sau revizuite pe baza schiței și a materialelor autorului. Autorul a verificat și își asumă integral conținutul final, conform Ghidului de utilizare a instrumentelor de IAG al ASE, aprobat prin HS nr. 38/25.03.2026.

# Structura capitolului

- Săptămâna 3: limbajul și pagina
  1. Limbajul JavaScript: tipuri, variabile, vectori
  2. Funcții: declarare, funcții săgeată, closure, map / filter / find / reduce
  3. Obiecte: proprietăți, metode, JSON, clase
  4. DOM: nodul ca obiect, regăsire, manipulare noduri, atribute și CSS
  5. Evenimente: ascultători, obiectul `event`; structura unei aplicații: model, afișare, evenimente; propagare și delegare
- Săptămâna 4: timpul și serverul
  6. Execuție asincronă: bucla de evenimente, temporizare, `Promise`, `async` / `await`
  7. Comunicarea cu serverul: Fetch API, JSON; XMLHttpRequest (opțional)

# Limbajul JavaScript



- Caracteristici
  - Dinamic
  - Limbaj funcțional
  - Bazat pe obiecte
- Utilizare
  - Manipulare noduri DOM
  - Evenimente
  - Procesare
  - Comunicare la distanță

# Modalități de includere în HTML



- Prin intermediul tag-ului `<script>`
- **A) Fișier extern**

```
<script src="lib/test.js"></script>
```

- **B) În cadrul paginii**

```
<script>
  // cod JavaScript …
</script>
```

- Execuție directă în fereastra de consolă
- Afișare mesaje: `console.log(valori);`

# Tipuri de date



- Tipuri de date de bază
  - **Number**: `-3.14`, `6`, `2`
  - **Boolean**: `true`, `false`
  - **String**: `"test"`
  - **null**, **undefined**
  - **Symbol**
  - **Object**: `{ nume: "Ana", varsta: 7 }` – referință
- Obiecte speciale
  - Function: `function f() {…}`
  - Array: `[1, 2, "trei"]`

# Variabile și expresii



- Declarare
  - `let`: `let b = "test"; let c;` – block scope; valoarea se poate schimba
  - `const`: `const d = "test";` – block scope; referința nu se mai poate schimba
  - Prin atribuire directă, fără declarare (**nerecomandat**): `a = 8;` – creează o variabilă globală
  - `var` – forma veche, cu function scope; nu se mai folosește
- Exemple:

```
let b = "test"; b = 1.5;                // același nume, alt tip
const c = 7;                             // c = 8 ar fi o eroare
const v = [1, 2, 3]; v[0] = 5;           // conținutul se schimbă, referința nu
```

# Variabile și expresii



- Scope
  - Global sau local
  - Function vs block based
- Evaluare expresii
  - Operatori similari cu C# (în plus `===` – egalitate strictă)
- Exemple:

```
a += 7          a === 10          a < b && b > c
a > 10          a++               a = "Ana" + "-" + "Maria"
```

- Utilizare *template strings*: `` `Numele este ${prenume} ${nume}.` ``

# Vectori – obiectul Array



- Inițializare: `let v = [];` sau `let v = [1, "Ion"];`; acces: `v[0]`, `v[1] = 23`; dimensiune: `v.length`

| Metodă | Efect | Rezultat |
|---|---|---|
| `push(valoare)` | adaugă la sfârșit | noua lungime |
| `pop()` | extrage ultimul element | elementul extras |
| `indexOf(valoare)` | caută o valoare | poziția sau -1 |
| `sort()`, `sort((a, b) => a - b)` | sortează pe loc; fără comparator, ca text | același vector |
| `slice(start, sfârșit)` | copiază un sub-vector; `sfârșit` exclusiv | vector nou |
| `splice(start, câte, val1, …)` | șterge sau înlocuiește pe loc | elementele șterse |

```
const v = [10, 9, 1];
v.sort();                    // → [1, 10, 9]: compară ca text
v.sort((a, b) => a - b);     // → [1, 9, 10]
v.slice(0, 2);               // → [1, 9]; v rămâne [1, 9, 10]
for (const x of v) console.log(x);   // 1  9  10
```

# Funcții



- Declarare:

```
function suma(a, b) { return a + b; }            // function declaration
const produs = function(a, b) { return a * b; } // function expression
```

- Parametri: valorile simple se transmit prin valoare; obiectele prin referință (funcția poate modifica obiectul primit)
- Parametri în număr variabil: `function suma(...numere) { … }` (*rest parameters*)
- Apel – `nume_funcție(parametri)` sau `expresie(parametri)`:

```
const rezultat = suma(7, 3);                      // → 10
const test = (function(val) { return val + 1; })(4);   // → 5
```

- Closure – vezi https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures

# Funcții săgeată și closure



- Funcția săgeată (*arrow function*): `(parametri) => expresie` sau `(parametri) => { instrucțiuni }`
  - Un singur parametru: parantezele pot lipsi; corpul fără acolade întoarce valoarea expresiei
  - Nu are `this` propriu: îl păstrează pe cel din funcția în care a fost scrisă

```
const dublu = x => x * 2;
const suma = (a, b) => a + b;
const salut = nume => { console.log(`Salut, ${nume}!`); };
dublu(4);          // → 8
salut("Ana");      // Salut, Ana!
```

- **Closure**: o funcție reține variabilele din locul în care a fost creată, chiar după ce acel loc și-a încheiat execuția

```
function contor() {
  let n = 0;
  return () => ++n;          // reține n
}
const c = contor();
c(); c(); c();               // → 1, 2, 3
```

# map, filter, find, reduce



- Metode ale vectorilor care primesc o funcție și o aplică fiecărui element, fără `for`

| Metodă | Efect | Rezultat |
|---|---|---|
| `map(f)` | aplică `f` fiecărui element | vector nou, aceeași lungime |
| `filter(f)` | păstrează elementele pentru care `f` întoarce `true` | vector nou, mai scurt |
| `find(f)` | primul element pentru care `f` întoarce `true` | elementul sau `undefined` |
| `reduce(f, inițial)` | acumulează: `f(acumulator, element)` | o singură valoare |
| `forEach(f)` | apelează `f` pentru fiecare element | nimic |

```
const persoane = [{ nume: "Popescu", varsta: 30 }, { nume: "Ionescu", varsta: 17 }];
persoane.map(p => p.nume);                    // → ["Popescu", "Ionescu"]
persoane.filter(p => p.varsta >= 18);         // → [{ nume: "Popescu", varsta: 30 }]
persoane.find(p => p.nume === "Ionescu");     // → { nume: "Ionescu", varsta: 17 }
persoane.reduce((s, p) => s + p.varsta, 0);   // → 47
```

# Obiecte



- Obiect = colecție de proprietăți (perechi *nume* = *valoare*)
- Declarare: literal – `let ob = { nume: "Ana", varsta: 7 }`
- Accesare proprietăți
  - Citire: `let v = ob.varsta;` sau `let v = ob["varsta"];`
  - Parcurgere: `for (const cheie in ob) { console.log(cheie, ob[cheie]); }`
  - Modificare / adăugare: `ob.varsta = 8; ob["varsta"] = 8; ob.clasa = 2;`
  - Ștergere: `delete ob.varsta;`
- Metode
  - adăugare: `ob.test = function() { console.log("test"); }`
  - `this`: `ob.afisare = function() { console.log("Nume: " + this.nume); }`
  - Apel: `ob.afisare();`

# JSON



- **JSON** (JavaScript Object Notation): format text pentru date, cu sintaxa literalilor JavaScript
  - Obiecte `{ }`, vectori `[ ]`, șiruri între ghilimele duble, numere, `true` / `false` / `null`
  - Numele proprietăților stau obligatoriu între ghilimele duble; fără funcții, fără comentarii
  - Formatul în care se trimit și se primesc date de la server (`response.json()`, secțiunea *Server*)
- `JSON.stringify(obiect)` – din obiect în text; `JSON.parse(text)` – din text în obiect

```
const persoana = { nume: "Popescu", telefon: "0234" };
const text = JSON.stringify(persoana);
console.log(text);                    // {"nume":"Popescu","telefon":"0234"}
const copie = JSON.parse(text);
console.log(copie.nume);              // Popescu
copie === persoana;                   // → false: alt obiect, același conținut
```

# Constructori și moștenire

- Funcțiile JavaScript pot fi utilizate pentru construirea de obiecte
  - Exemplu: `function Persoana(nume) { this.nume = nume; this.varsta = 1; }`
  - Apel: `const ob2 = new Persoana("Maria");`
- `prototype`
  - Proprietate a funcției constructor
  - Definește proprietățile disponibile în toate obiectele (instanțele create)
  - Exemplu: `Persoana.prototype.afisare = function() { console.log("Nume: " + this.nume); };`
  - Folosit și pentru implementarea moștenirii:

```
function Student() { this.facultate = "CSIE"; }
Student.prototype = new Persoana("-");
const s = new Student(); s.afisare();
```

# Clase

- Limbajul JavaScript permite utilizarea cuvântului cheie `class` pentru definirea funcțiilor constructor în forma:

```
class NumeClasa {
  constructor(parametri) {
    this.proprietate1 = valoare;
    this.proprietate2 = valoare;
  }
  metoda1() { ... }
  metoda2() { ... }
  get numeProp() { ... }
  set numeProp(value) { ... }
}
```

- Moștenirea se implementează folosind cuvântul cheie `extends`:

```
class Derivata extends NumeClasa { /* definire metode */ }
```

# Nodul ca obiect



- Arborele DOM este construit din obiecte JavaScript; fiecare nod este un obiect derivat din `Node`
- Rădăcina este `document`; `document.documentElement` este elementul `html`, cu copiii `document.head` și `document.body`
- Proprietăți de navigare:

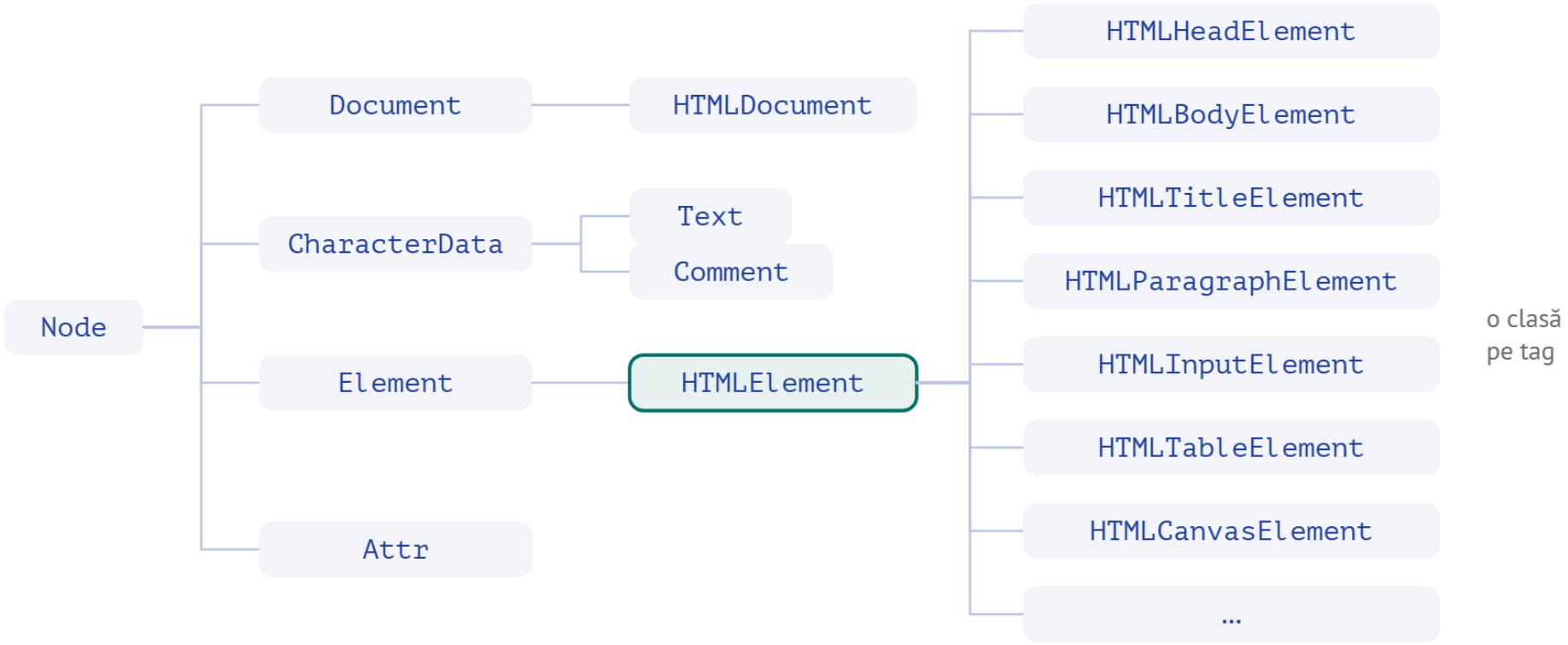
| Proprietate | Valoare |
|---|---|
| `childNodes` | toți copiii, inclusiv nodurile text și comentariile |
| `children` | doar copiii de tip element |
| `parentNode`, `parentElement` | părintele |
| `firstChild`, `lastChild`, `nextSibling`, `previousSibling` | vecinii, inclusiv nodurile text |
| `firstElementChild`, `nextElementSibling` | vecinii de tip element |

- Nodurile au metode și proprietăți specifice în funcție de tag-ul HTML utilizat pentru construirea nodului

# Ierarhia de clase DOM



Fiecare nod este un obiect dintr-o ierarhie de clase: elementele HTML derivă din HTMLElement, iar fiecare tag are clasa lui, cu proprietățile și metodele ei.



# Regăsire noduri



| Metodă | Caută | Rezultat |
|---|---|---|
| `document.getElementById(id)` | după identificator | un element sau `null` |
| `element.querySelector(selector)` | primul descendent care respectă selectorul CSS | un element sau `null` |
| `element.querySelectorAll(selector)` | toți descendenții care respectă selectorul | o listă (`NodeList`) |
| `element.closest(selector)` | cel mai apropiat strămoș, inclusiv elementul însuși | un element sau `null` |
| `element.getElementsByClassName(clasă)` | după clasă | o colecție vie |
| `element.getElementsByTagName(tag)` | după tag | o colecție vie |

```
const titlu = document.querySelector("h1");              // primul h1
const celule = document.querySelectorAll("tbody td");   // toate celulele din tbody
celule.length;                                           // → 6
const rand = celule[0].closest("tr");                    // rândul celulei
```

# Manipulare noduri



| Metodă / proprietate | Efect |
|---|---|
| `document.createElement(tag)`, `document.createTextNode(text)` | creează un nod nou, neatașat |
| `innerText`, `textContent` | textul nodului; la scriere, înlocuiește conținutul cu text |
| `innerHTML` | conținutul ca HTML; la scriere, textul este interpretat ca HTML |
| `parinte.append(copil)`, `parinte.prepend(copil)` | adaugă la sfârșit sau la început |
| `nod.before(nou)`, `nod.after(nou)`, `nod.replaceWith(nou)` | înainte, după, în loc |
| `nod.remove()`, `parinte.removeChild(copil)` | elimină |

```
const tbody = document.querySelector("tbody");
const rand = document.createElement("tr");
const celula = document.createElement("td");
celula.innerText = "Popescu";
rand.append(celula);
tbody.append(rand);            // în pagină apare un rând nou cu o celulă
tbody.innerHTML = "";          // golește corpul tabelului
```

# Manipulare atribute



- Accesare atribute:
  - `elem.numeAtribut` – accesare valoare atribut individual
  - `elem.attributes` – colecția de atribute
  - `elem.hasAttribute(name)` – verificare existență
  - `elem.getAttribute(name)` – obținere valoare
  - `elem.setAttribute(name, value)` – modificare valoare
  - `elem.removeAttribute(name)` – ștergere atribut
- Asociere date proprii:
  - Atribute HTML: `<tag data-denumire="valoare"></tag>`
  - Din JavaScript: `element.dataset.denumire`

# Manipulare CSS



- Accesare atribute CSS:
  - `elem.style.numeProprietate` – stilul inline; numele CSS se scriu *camelCase*: `background-color` devine `backgroundColor`
  - `elem.className` – string, corespunzător atributului `class` din HTML
  - `elem.classList` – obiect care permite manipularea listei de clase (metode `add` / `remove` / `toggle` / `contains`)
- Obținere coordonate element față de fereastră: `elem.getBoundingClientRect()`
- Exemplu:

```
let p = document.createElement("p");
p.innerText = "test";
document.querySelector("body").append(p);
p.style.backgroundColor = "red";
```

# Tratare evenimente



- Adăugare event handler:
  - Prin atribute HTML: `<element atributEveniment="cod JavaScript">...</element>`
    - Exemplu: `<a onclick="test()">test</a>`
  - Prin proprietăți nod: `element.proprietateEveniment = funcție;`
    - Exemplu: `document.getElementById("test").onclick = function() { console.log("un mesaj"); }`
  - **Prin metoda addEventListener:** `element.addEventListener(tip, funcție);`
    - Exemplu: `document.getElementById("test").addEventListener("click", function() { console.log("un mesaj"); });`
    - Mai multe funcții pentru același eveniment; funcția poate fi scoasă ulterior
- Eliminare event handler:
  - `element.removeEventListener(tip, funcție);`

# Tratare evenimente



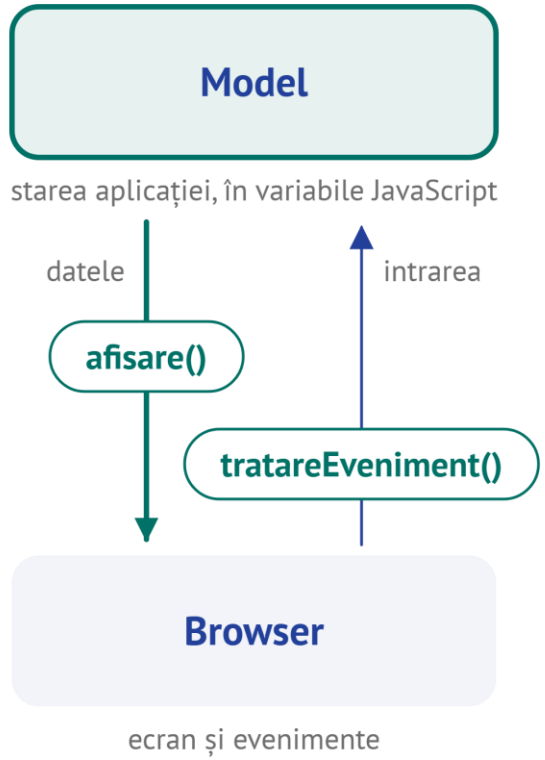
- Parametrul funcției – obiectul `event`
  - General: `target` (elementul pe care s-a produs evenimentul), `currentTarget` (elementul pe care este ascultătorul), `type`, `preventDefault()`
  - Tastatură: `key` (caracterul: `"a"`, `"Enter"`), `code` (tasta fizică: `"KeyA"`), `altKey`, `ctrlKey`, `shiftKey`
  - Mouse: `x`, `y`, `button`, `altKey`, `ctrlKey`, `shiftKey`
- Evenimente
  - General: `load` (window), `DOMContentLoaded` (document)
  - Tastatură: `keydown`, `keyup`
  - Mouse: `mouseenter`, `mouseleave`, `mousemove`, `mouseup`, `mousedown`, `click`, `dblclick`, `contextmenu`
  - Formular: `input`, `change`, `submit`

```
buton.addEventListener("click", e => {
    console.log(e.type, e.currentTarget.id);    // click btnAdauga
});
```

# Modelul general al unei aplicații



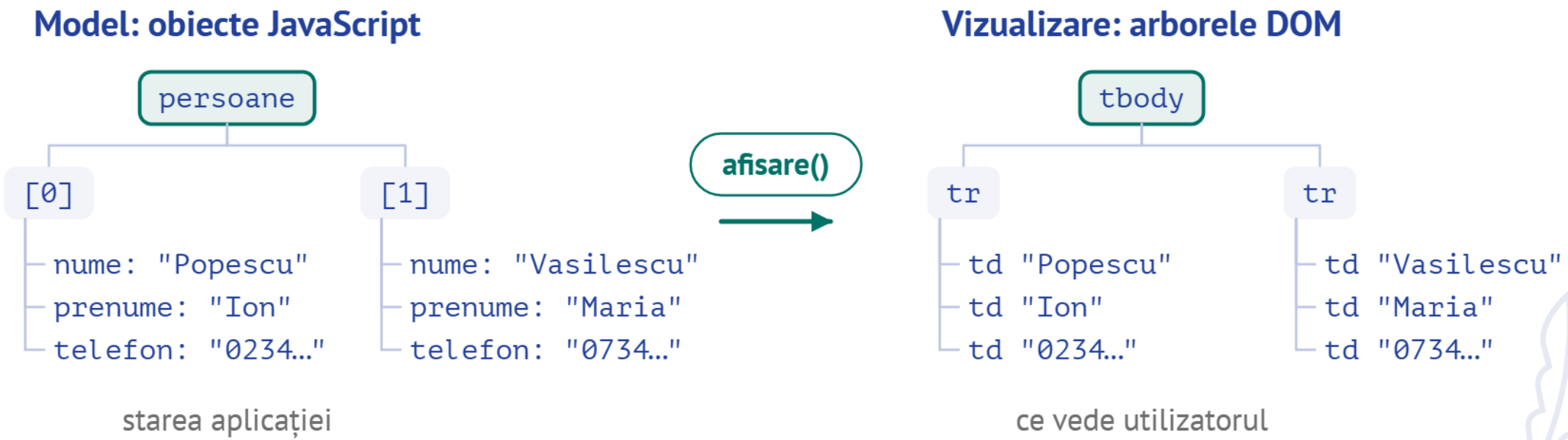
- **Modelul** este starea aplicației, păstrată în variabile JavaScript (valori, obiecte, vectori)
- `afisare()` construiește ecranul din model, niciodată invers: după apel, ecranul corespunde modelului
- **Tratarea evenimentelor** modifică modelul, apoi apelează `afisare()`; nu modifică ecranul direct
- Browserul apelează funcțiile; săgețile arată datele, nu apelurile
- Aceeași structură pentru orice aplicație: la animație (capitolul 9) se adaugă timpul, cu `actualizareModel()`



# Structura unei aplicații



Starea aplicației stă în **model**, un arbore de obiecte JavaScript; funcția `afisare()` construiește din el arborele DOM pe care îl vede utilizatorul, niciodată invers.



# Modelul și afișarea în cod



- Modelul: variabile JavaScript; afișarea: golește `tbody` și îl reconstruiește din model

```
// 1. Model
let persoane = [
    { nume: "Popescu", prenume: "Ion", telefon: "023423222" },
    { nume: "Vasilescu", prenume: "Maria", telefon: "073423222" }
];
// 2. Afișare: tot ce este pe ecran vine din model
function afisare() {
    const tbody = document.querySelector("tbody");
    tbody.innerHTML = "";
    for (const persoana of persoane) {
        const rand = document.createElement("tr");
        for (const valoare of [persoana.nume, persoana.prenume, persoana.telefon]) {
            const celula = document.createElement("td");
            celula.innerText = valoare;
            rand.append(celula);
        }
        tbody.append(rand);
    }
    document.querySelector("tfoot td").innerText = `Număr persoane: ${persoane.length}`;
}
```

# Aplicația completă



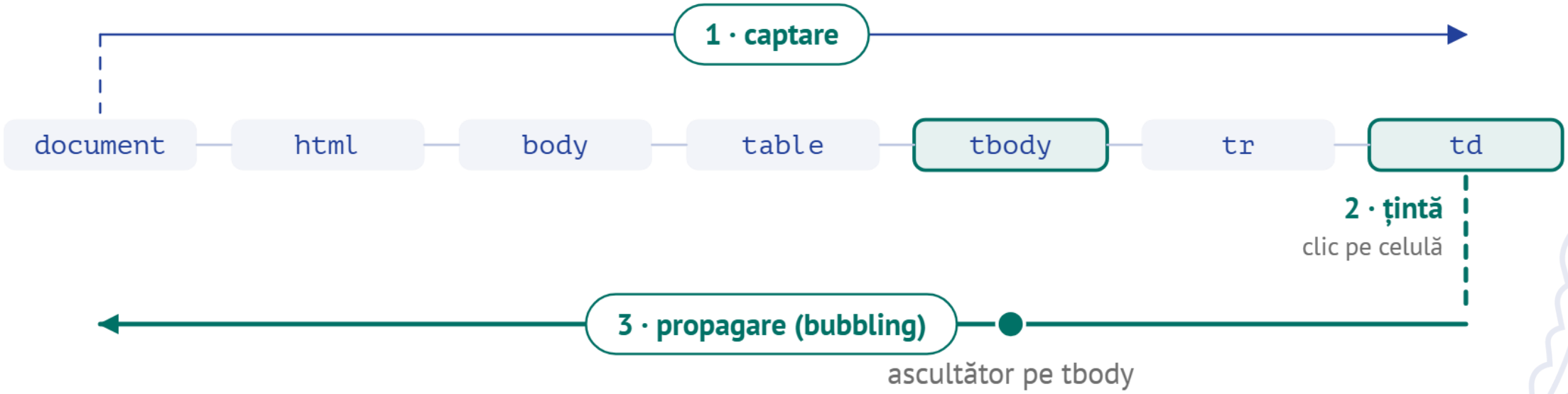
- Trei roluri: **modelul** ține starea, **afisare()** construiește ecranul din model, **tratarea evenimentelor** modifică modelul și apoi apelează `afisare()`
- Browserul apelează funcțiile; codul nu modifică niciodată tabelul direct

```
// 3. Tratare evenimente: din formular în model, apoi afișare
function adaugare() {
    persoane.push({
        nume: document.querySelector("#nume").value,
        prenume: document.querySelector("#prenume").value,
        telefon: document.querySelector("#telefon").value
    });
    afisare();
}
// 4. Pornire: după ce arborele DOM este construit
function aplicatie() {
    document.querySelector("#btnAdauga").addEventListener("click", adaugare);
    afisare();
}
document.addEventListener("DOMContentLoaded", aplicatie);
```

# Propagarea evenimentelor



Un eveniment nu rămâne pe elementul pe care s-a produs: coboară de la `document` până la țintă, apoi urcă înapoi, iar ascultătorii strămoșilor îl primesc la urcare.



# Propagarea evenimentelor



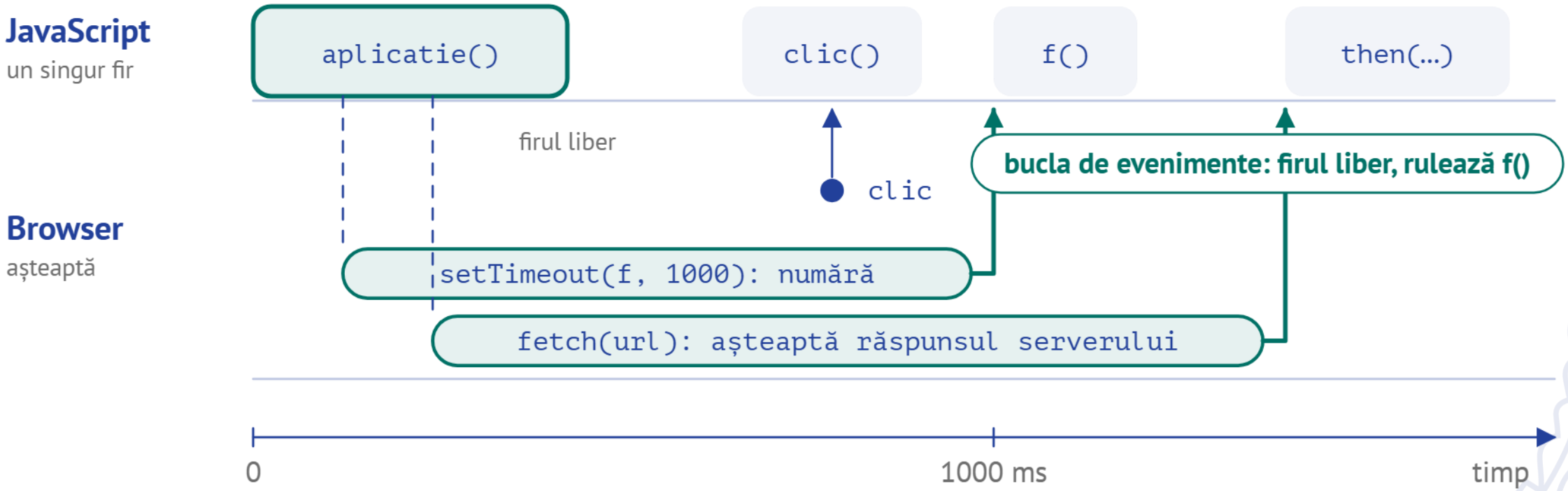
- Propagarea permite un singur ascultător pe un strămoș, în locul unui ascultător pe fiecare element
  - `event.target` spune pe ce element s-a produs evenimentul; `closest` urcă la elementul care interesează
  - Funcționează și pentru elementele adăugate ulterior, de exemplu rândurile create de `afisare()`
- `event.stopPropagation()` oprește urcarea; `preventDefault()` oprește acțiunea implicită (trimiterea formularului, urmarea link-ului)

```
const tbody = document.querySelector("tbody");
tbody.addEventListener("click", e => {
    const rand = e.target.closest("tr");    // celula apăsată → rândul ei
    if (!rand) return;
    console.log(rand.rowIndex);             // 1 pentru primul rând din tbody
    rand.classList.toggle("selectat");
});
```

# Bucla de evenimente



JavaScript rulează pe un singur fir: `setTimeout` și `fetch` pornesc o așteptare în browser, iar funcția dată rulează mai târziu, când firul este liber.



# Programare execuție funcție



- A) Execuție o singură dată după un interval de timp specificat
  - Programare pornire: `const id = setTimeout(funcție, durata[, param1, param2, ...]);`
  - Oprire: `clearTimeout(id);`
- B) Execuție repetată la un interval de timp fix
  - Programare pornire: `const id = setInterval(funcție, durata[, param1, param2, ...]);`
  - Oprire: `clearInterval(id);`
- Observație: duratele sunt exprimate în milisecunde; funcția nu rulează înainte de a se goli stiva, deci durata este un minim
- Exemplu (`04_03_slideshow.html`): un slide la fiecare secundă, oprit la clic

```
let timerId = setInterval(() => {
    position = position >= slides.length - 1 ? 0 : position + 1;
    afisare();
}, 1000);
document.querySelector("p").addEventListener("click", () => clearInterval(timerId));
```

# Execuție asincronă – obiecte Promise



- Obiecte `Promise` – încapsulează o operație asincronă și callback-urile asociate
- Construire: o funcție care întoarce `new Promise(executor)`
  - funcția *executor* primește două funcții, `resolve` și `reject`: execută operația, apoi apelează `resolve(valoareRezultat)` la succes sau `reject(eroare)` la eșec
- Utilizare: se apelează funcția și se obține obiectul `Promise`
  - metoda `then` primește funcția de executat după încheierea operației; dacă aceasta întoarce un alt `Promise`, apelurile `then` se înlănțuie
  - metoda `catch` primește funcția de executat la eșec, pentru întreg lanțul

```
afisare("Ana")
    .then(() => afisare("are"))
    .then(() => afisare("mere"))
    .catch(eroare => console.error(eroare));
```

# JavaScript – execuție asincronă



- Instrucțiuni dedicate pentru obiecte `Promise`:
  - `async function`: declară o funcție asincronă care întoarce un obiect `Promise`; permite utilizarea operatorului `await` în corpul funcției
  - `await promisiune`: suspendă funcția `async` până când promisiunea se încheie și întoarce valoarea ei; restul paginii nu este blocat; la eșec, aruncă eroarea, prinsă cu `try` / `catch`
- Exemplu:

```
function afisare(mesaj) {
  return new Promise(resolve => {
    setTimeout(() => { console.log(mesaj); resolve(`Am afișat '${mesaj}'.`); }, 1000);
  });
}

async function test() {
  let valoare = await afisare("Ana");
  console.log(valoare);                    // Am afișat 'Ana'.
  valoare = await afisare("are");
  valoare = await afisare("mere");
}
test();
```

- Resurse: https://javascript.info/async

# Comunicarea cu serverul – Fetch API



- Funcție: `fetch(URL[, {options}])` – întoarce un obiect de tip `Promise`
- Opțiuni:
  - `method` – `'GET'` (implicit), `'POST'`, `'PUT'`, `'DELETE'`
  - `headers` – obiect care conține header-ele de trimis către server sub formă de proprietăți
  - `body` – conținutul pentru cereri de tip POST sau PUT (string, `FormData`, ...)
- Obiectul răspuns:
  - proprietăți pentru determinarea rezultatului (`ok`, `status`, `statusText`)
  - `headers` – obiect care permite citirea datelor din header-ul HTTP
  - Metode pentru citirea conținutului (`text()`, `json()`, `formData()`, ...) – întorc un obiect de tip `Promise`
- Promisiunea eșuează doar la eroare de rețea; un răspuns 404 sau 500 este un răspuns primit, cu `ok` egal cu `false`

# Comunicarea cu serverul – Fetch API



- Exemplu: citirea agendei dintr-un fișier JSON de pe server (`04_05_fetch.html`)

```
async function app() {
  const raspuns = await fetch('media/agenda.json');
  if (!raspuns.ok) {
    console.error(`Eroare ${raspuns.status}: ${raspuns.statusText}`);
    return;
  }
  const persoane = await raspuns.json();   // text JSON → vector de obiecte
  console.log(persoane[0].nume);           // Vasilescu
}
app();
```

- Trimiterea unui obiect: `fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(persoana) })`
- `fetch` are nevoie de un server (Live Server): dintr-un fișier deschis cu dublu clic, cererea este refuzată
- Resurse: https://developer.mozilla.org/en-US/docs/Web/API/fetch

# Comunicarea cu serverul – XMLHttpRequest

OPȚIONAL

Server

- Prin obiecte de tip `XMLHttpRequest`
- Permite crearea, transmiterea și recepția de cereri HTTP(S)
- **Construire** obiect cerere: `const cerere = new XMLHttpRequest();`
- **Funcții**:
  - `open(method, url)` – inițializează un obiect `XMLHttpRequest`
  - `setRequestHeader(header, value)` – permite transmiterea de valori în header-ul cererii
  - `send(data)` – începe procesarea asincronă a cererii
  - `abort()` – anulează o cerere în desfășurare
  - `getResponseHeader(header)` – citește un header din răspunsul primit

# Comunicarea cu serverul – XMLHttpRequest

- **Proprietăți**:
  - `readyState` – întreg care indică starea curentă a cererii (0 – netrimisă, …, 4 – done)
  - `response` – răspunsul primit de la server de tipul indicat de `responseType`
  - `status`, `statusText` – codul HTTP pentru răspuns în format numeric și text
- **Evenimente** – `addEventListener`:
  - `load` – cererea a fost executată și a fost primit răspunsul
  - `readystatechange` – s-a modificat starea curentă a obiectului
  - `progress` – avansare operație
  - `abort` – cererea a fost anulată de client
  - `timeout` – a expirat timpul alocat
  - `error` – cererea s-a încheiat cu o eroare